

Apex Bank — Microservices Portal

Complete Technical Reference, API Specifications, Role-Based Access Control & Startup Guide

ARCHITECTURE	FRONTEND	DATABASE	DISCOVERY
Spring Cloud Microservices	React 19 + TypeScript + Vite	MySQL 8.0 (InnoDB)	Netflix Eureka (:8761)

1. System Architecture & Topology

The system is built as a distributed, service-oriented architecture consisting of five backend Spring Boot services, an Apache Kafka messaging broker, a central MySQL database, and a high-performance single-page application (SPA) built with React and Vite.

MICROSERVICE	PORT	APPLICATION ID	ROLE & KEY RESPONSIBILITIES
Eureka Server	:8761	eureka-server	Central service registry and heartbeat coordinator for dynamic discovery.
API Gateway	:8090	api-gateway	Spring Cloud Gateway (Reactive WebFlux) routing requests to services via Eureka.
Banking Service	:8081	banking-service	Core banking: JWT auth, bank accounts, audit ledger, and transactions.
Payroll Service	:8082	payroll-service	Enterprise payroll management with Kafka asynchronous event streaming.
AI Chatbot Service	:8083	ai-chatbot	AI financial assistance resolving natural-language operations queries.
Vite Frontend	:5173	frontend	Modular client SPA with dual Light/Dark themes and Live/Mock fallback modes.
MySQL Database	:3306	banking_payroll	ACID relational storage for users, roles, accounts, transactions, and staff.

2. Seed User Accounts & Credentials (MySQL)

These default credentials are pre-seeded in the database (`banking_payroll.users`) and encrypted with **BCrypt**.

ROLE	USERNAME	CORPORATE EMAIL	PASSWORD	SYSTEM PERMISSIONS
ROLE_ADMIN	admin	admin@bank.com	Admin@123	Full System Administrator: Create HR accounts, manage staff directory, open/close bank

ROLE	USERNAME	CORPORATE EMAIL	PASSWORD	SYSTEM PERMISSIONS
				accounts, post transactions, export CSV, and view cluster topology.
ROLE_HR	hr	hr@bank.com	Hr@123456	Human Resources Manager: Manage staff directory (add/edit employees, salaries, designations), AI assistant, and cluster health. Restricted from banking accounts and ledger.
ROLE_EMPLOYEE	john_fintech	john_fintech@gmail.com	Password1234	Standard Banking Client: View assigned bank accounts, execute deposits/withdrawals/transfers, view audit ledger, export CSV, and AI assistant. Restricted from staff directory.
ROLE_EMPLOYEE	testuser	testuser@gmail.com	Test@123	Standard employee secondary account.

One-Click Demo Fill: On the Login page (</login>), click the **Admin**, **HR Mgr**, or **Staff** shortcut buttons to automatically populate these credentials.

3. Technologies Used Across the Stack

A. Frontend Application (Directory: `c:\Users\saisa\Desktop\Poc Implemented\frontend`)

- React 19 & TypeScript:** Component modularity, strict typing, and high rendering efficiency.
- Vite 8.3 Bundler:** Native ESM-powered development server with instant HMR and production rollups.
- TanStack React Query v5:** Server-state synchronization, automatic caching, garbage collection, and optimistic mutation rollbacks.
- Zustand:** Lightweight global client state for auth tokens, demo role switching, theme preference, and mock fallback toggling.
- React Router v7:** Declarative client routing with protected routes and role-based redirect guards.
- Axios:** Robust HTTP client with automatic JWT Bearer token request interceptor and error status handling.
- Pure Vanilla CSS Design System:** Complete dual-theme architecture (Modern Fintech Light + Obsidian Dark) inspired by Stripe and Mercury Bank, with 0% heavy CSS framework overhead.
- Lucide React:** Clean, crisp vector iconography.

B. Backend Microservices

- Java 21 / 24 & Spring Boot 4.1.0:** High-throughput reactive and servlet runtimes.
- Spring Cloud 2025.1.2:** Microservices coordination, client-side load balancing, and dynamic routing.
- Netflix Eureka Server & Client:** Dynamic service registration, heartbeat validation, and instance cataloging.
- Spring Cloud Gateway (WebFlux):** Asynchronous reverse proxy and request forwarding to Eureka instances.
- Spring Security 6 & JWT:** Stateless authentication with HMAC-SHA256 encrypted JWT Bearer tokens (1-hour validity).
- Spring Data JPA & Hibernate:** Relational object-relational mapping, transactional integrity, and automated schema validation.
- MySQL 8.0 Community Server:** Transactional relational database engine running under Windows Service `MYSQl80` .
- Apache Kafka:** Distributed event streaming bus for asynchronous inter-service workflows.
- SpringDoc OpenAPI / Swagger UI:** Interactive API schema exploration and documentation.

4. Role-Based Access Control (RBAC) Matrix

The table below summarizes page and endpoint access rules enforced across both the React frontend and Spring Security filters:

FEATURE / VIEW	URL ROUTE	ADMIN	HR MGR	EMPLOYEE	GUEST / PUBLIC
Login & Register	<code>/login</code>	Allowed	Allowed	Allowed	Allowed
Executive Overview	<code>/dashboard</code>	Allowed	Allowed	Allowed	Redirect to Login
Bank Accounts & 3D Cards	<code>/accounts</code>	Allowed	Restricted	Allowed	Redirect to Login
Open New Account	Modal on <code>/accounts</code>	Allowed	Restricted	Allowed	Redirect to Login

FEATURE / VIEW	URL ROUTE	ADMIN	HR MGR	EMPLOYEE	GUEST / PUBLIC
Close Account (Zero Txs)	Delete button on <code>/accounts</code>	✓ Allowed	✗ Restricted	✗ Restricted	✗ Redirect to Login
Transaction Ledger	<code>/transactions</code>	✓ Allowed	✗ Restricted	✓ Allowed	✗ Redirect to Login
Post Deposit / Withdrawal	Modal on <code>/transactions</code>	✓ Allowed	✗ Restricted	✓ Allowed	✗ Redirect to Login
Export Ledger to CSV	Button on <code>/transactions</code>	✓ Allowed	✗ Restricted	✓ Allowed	✗ Redirect to Login
Staff Directory	<code>/employees</code>	✓ Allowed	✓ Allowed	✗ Redirect to Dashboard	✗ Redirect to Login
Add / Edit Staff Member	Modal on <code>/employees</code>	✓ Allowed	✓ Allowed	✗ Restricted	✗ Redirect to Login
Create HR Account	Modal on <code>/employees</code>	✓ Allowed	✗ Restricted	✗ Restricted	✗ Redirect to Login
AI Financial Assistant	<code>/assistant</code>	✓ Allowed	✓ Allowed	✓ Allowed	✗ Redirect to Login
Floating AI Assistant Drawer	Bottom-Right Popup	✓ Allowed	✓ Allowed	✓ Allowed	✗ Restricted
Cluster Topology & Health	<code>/health</code>	✓ Allowed	✓ Allowed	✓ Allowed	✗ Redirect to Login

5. Complete REST API Specifications

A. Authentication Endpoints (Banking Service — Port 8081)

1. Authenticate with Email & Password (Login)

```
POST http://localhost:8081/auth/login
Content-Type: application/json

Request:
{
  "email": "admin@bank.com",
  "password": "Admin@123"
}

Response (200 OK):
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbGlzInJvbGUiOiJST0xFX0FETU10Ii... [JWT TOKEN]"
}
```

2. Register New User (Public)

```
POST http://localhost:8081/auth/register
Content-Type: application/json

Request:
{
  "username": "alex_fintech",
  "email": "alex@bank.com",
  "password": "Password123!"
}

Response (200 OK):
"User registered successfully"
```

3. Create HR Account (Admin Only)

```
POST http://localhost:8081/auth/create-hr
Authorization: Bearer <ADMIN_JWT>
Content-Type: application/json

Request:
{
  "username": "hr_lead",
  "email": "hr_lead@bank.com",
  "password": "SecurePassword123"
}

Response (200 OK):
"HR user created successfully"
```

B. Bank Account Endpoints (Banking Service — Port 8081)

Requires `Authorization: Bearer <TOKEN>` (Roles: ADMIN, EMPLOYEE)

METHOD	URL	DESCRIPTION
GET	<code>http://localhost:8081/accounts</code>	List all active bank accounts from MySQL.
GET	<code>http://localhost:8081/accounts/{id}</code>	Fetch a single account by primary key ID.
POST	<code>http://localhost:8081/accounts</code>	Open a new bank account.
PUT	<code>http://localhost:8081/accounts/{id}</code>	Update account details or associated employee ID.
DELETE	<code>http://localhost:8081/accounts/{id}</code>	Close an account (only allowed if it has 0 transactions).

Sample Account Creation Request (`POST /accounts`):

```
POST http://localhost:8081/accounts
Authorization: Bearer <TOKEN>
Content-Type: application/json

{
  "accountNumber": "ACC10003",
  "accountType": "SAVINGS",
  "balance": 15000.00,
  "employeeId": 1
}
```

C. Transaction Endpoints (Banking Service — Port 8081)

Requires `Authorization: Bearer <TOKEN>` (Roles: ADMIN, EMPLOYEE)

METHOD	URL	DESCRIPTION
GET	<code>http://localhost:8081/transactions</code>	Retrieve the full financial audit ledger from MySQL.
GET	<code>http://localhost:8081/transactions/{id}</code>	Fetch single transaction details.
POST	<code>http://localhost:8081/transactions</code>	Execute deposit/withdrawal; automatically increments/decrements account balance.
DELETE	<code>http://localhost:8081/transactions/{id}</code>	Delete an audit entry from ledger.

Sample Transaction Request (`POST /transactions`):

```
POST http://localhost:8081/transactions
Authorization: Bearer <TOKEN>
Content-Type: application/json

{
  "accountId": 2,
  "amount": 1250.00,
  "transactionType": "DEPOSIT",
}
```

```
"remarks": "Executive Q3 Bonus Allocation"
}
```

D. Staff & Employee Endpoints (Banking Service — Port 8081)

Requires `Authorization: Bearer <TOKEN>` (Roles: ADMIN, HR)

METHOD	URL	DESCRIPTION
GET	<code>http://localhost:8081/employees</code>	List all staff members and payroll records.
GET	<code>http://localhost:8081/employees/{id}</code>	Fetch employee profile by ID.
POST	<code>http://localhost:8081/employees</code>	Add a new staff record.
PUT	<code>http://localhost:8081/employees/{id}</code>	Update employee designation, department, or salary.
DELETE	<code>http://localhost:8081/employees/{id}</code>	Remove employee record.

E. AI Chatbot API (AI-Chatbot Service — Port 8083)

```
POST http://localhost:8083/api/chat
Content-Type: application/json

Request:
{
  "message": "What is my account balance and how do I deposit funds?",
  "userId": "admin"
}

Response (200 OK):
{
  "response": "AI Response: What is my account balance and how do I deposit funds?",
  "status": "SUCCESS"
}
```

F. Eureka Discovery Registry (Eureka Server — Port 8761)

Web Dashboard: `http://localhost:8761/` (Visual summary of all registered instances).

REST API: GET `http://localhost:8761/eureka/apps` with header `Accept: application/json` .

6. Step-by-Step Guide: How to Run Frontend & Backend

Prerequisites: Ensure you have **Java 21 or 24** (`java -version`), **Maven 3.9+** (`mvn -v`), **Node.js 18+** (`node -v`), and **MySQL 8.0** installed.

1 Verify MySQL Database Service

Ensure that MySQL is running on port `3306` with database `banking_payroll` :

```
# Check MySQL Windows service
Get-Service *mysql*

# Verify tables exist
mysql -u root -p"Sanjay@2004" -e "USE banking_payroll; SHOW TABLES;"
```

2 Start Eureka Discovery Server (Port 8761)

Always start Eureka Server first so microservices can register immediately on boot:

```
cd "c:\Users\saisa\Desktop\Poc Implemented\Eureka-Server"
mvn spring-boot:run
```

Verify in browser: Open `http://localhost:8761` to see the Eureka Dashboard.

3 Start Banking Service (Port 8081)

Runs the core banking ledger, JWT auth, and MySQL data access:

```
cd "c:\Users\saisa\Desktop\Poc Implemented\Banking-Service"
mvn spring-boot:run
```

4 Start Payroll Service (Port 8082)

Runs employee payroll management and registers with Eureka:

```
cd "c:\Users\saisa\Desktop\Poc Implemented\Payroll-Service"
mvn spring-boot:run
```

5 Start AI Chatbot Service (Port 8083)

Runs the AI conversational assistant service:

```
cd "c:\Users\saisa\Desktop\Poc Implemented\AI-Chatbot_Geethika"
```

```
mvn spring-boot:run
```

6 Start Spring Cloud API Gateway (Port 8090)

Provides unified reactive proxying and dynamic routing to Eureka instances:

```
cd "c:\Users\saisa\Desktop\Poc Implemented\API-Gateway"  
mvn spring-boot:run
```

(Note: Configured to port 8090 to avoid collision with Oracle TNS Listener on port 8080).

7 Start Vite React Frontend (Port 5173)

Launch the React single-page portal:

```
cd "c:\Users\saisa\Desktop\Poc Implemented\frontend"  
npm install    # (First time only)  
npm run dev
```

Open <http://localhost:5173> in any web browser!

7. Testing & Verification Checklist

1. **Open Portal:** Navigate to <http://localhost:5173/login> .
2. **Ensure Live Mode:** Verify the top badge displays [Live Backend \(:8081\)](#) . (If it displays Mock Mode, click it once to switch).
3. **Sign In as Admin:** Click **Admin** shortcut button (or enter [admin@bank.com](#) / [Admin@123](#)).
4. **Verify Real MySQL Data:**
 - Go to **Bank Accounts** (</accounts>): verify [ACC10001](#) and [ACC10002](#) are loaded from MySQL.
 - Go to **Transaction Ledger** (</transactions>): verify transaction #3 ("Executive Q3 Bonus Allocation") is displayed.
 - Click **Post Transaction**: submit a deposit of \$500.00 and watch the balance automatically increment.
5. **Test AI Assistant:** Go to </assistant> and send any query to receive live responses from Port 8083.
6. **Inspect Cluster Topology:** Go to </health> to inspect live ping status across Eureka, Gateway, Banking, Payroll, and AI nodes.
7. **Toggle Themes:** Click the **Sun/Moon icon** in the top navbar to switch between Crisp Light and Obsidian Dark modes.